

its goal, with Swift voted as the “most loved programming language” in a StackOverflow poll last year. Considering that Swift 2.2 onwards has been made open-source, Apple’s ambitious project of making Swift compatible with more platforms such as Windows, Raspberry PI, Android, Linux makes NOW the best time to join Apple’s developer community.

Obviously, the release of Swift has also been accompanied by the Objective C vs. Swift debate – is it necessary for any developer worth his salt to bother learning Objective C anymore? While the debate still rages on, one point has been agreed up unanimously - for a newbie programmer interested in creating an iOS, OSX and watchOS app, picking Swift over Objective-C seems like the sensible step forward. Considering the much steeper learning curve associated with Objective C, learning Swift is easier and provides results faster. And for the seasoned programmer, any objections over Swift’s compilation speed or performance specs can be laid to rest, simply because Swift is compiled to native ARM or Intel code, just like Objective-C. And it uses the same ARC (automatic reference counting) that Objective-C uses, instead of a garbage collector. This efficiency has allowed Apple to ship iOS devices with half the RAM of equivalent Android and Windows Phone devices, and still have excellent performance.

A brief history is in order before you jump onto the Swift bandwagon. While various releases are available for you to use, the latest, most stable release version of 2.2.2 is definitely preferred. A simple list of version differences might also help settle any questions you have about which version to start off with:

1. TRY/CATCH:

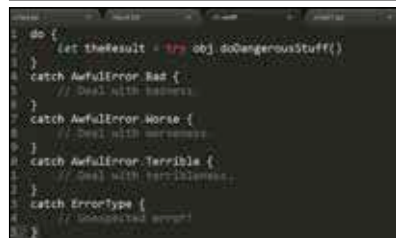
Up till now, the regular way to invoke error handling capabilities in Swift was to use the NSError object, as it is done in Objective C. However, that was more of a framework necessity,



```
enum ThirstyError: ErrorType {
    case NoWaterRemainingError
}

// 2
func thirstyWithError() throws {
    if water.isAvailable() {
        // drink!
    } else {
        // thirsty
        throw ThirstyError.NoWaterRemainingError
    }
}

func tryToDrink() {
    do {
        try thirstyWithError()
    } catch {
        print("Could not drink water! :(")
        return
    }
}
```



```
do {
    let theResult = try obj.dangerousStuff()
} catch AwwfulError.Bad {
    // Deal with badness
} catch AwwfulError.Worse {
    // Deal with worseness
} catch AwwfulError.Terrible {
    // Deal with terribleness
} catch ErrorType {
    // Unspecified error!
}
```